TriCoreFS: A Dual-Module File System Simulator and Organizer for Operating System Education

*Bridging Practical File Management with Core OS Concepts through Interactive Visualization

1st Vivekananda
Department of Computer Science
Rishihood University
Sonipat, India
pottabathini.v23csai@nst.rishihood.edu.in

2nd Aditya Kammati
Department of Computer Science
Rishihood University
Sonipat, India
kammati.a23csai@nst.rishihood.edu.in

3rd Charan Aditya
Department of Computer Science
Rishihood University
Sonipat, India
kolli.c23csai@nst.rishihood.edu.in

***Abstract*—File systems constitute one of the most critical components of modern operating systems, yet their internal mechanics remain opaque to most students and practitioners. This paper presents TriCoreFS, a dual-module educational system that bridges practical file management with core OS file system concepts. The first module, File Organiser, implements a real-world utility that recursively categorizes uploaded directories based on file type metadata, demonstrating hierarchical directory structures and file manipulation operations. It is further accessible as a published command-line interface (CLI) tool via NPM (`npx smart-folder-organiser`), extending its utility to terminal-based workflows. The second module, File System Simulator, provides interactive visualization of five allocation strategies—Contiguous, Linked, Indexed, UNIX Inode, and FAT-style—alongside inode structure modeling and fragmentation analysis. Built using React, Node.js, and browser-native APIs, TriCoreFS enables users to specify file sizes, select allocation strategies, and observe block-level disk behavior in real time. The system serves as both a practical automation tool and an educational platform, concretizing abstract OS concepts including block allocation, pointer structures, and storage efficiency trade-offs.**

***Index Terms*—File System, Block Allocation, Inode, Fragmentation, Contiguous Allocation, Linked Allocation, Indexed Allocation, UNIX Inode, FAT, Operating Systems Education, File Organiser, CLI Tool**

I. Introduction

File systems are the foundational layer through which operating systems manage persistent data. Despite their central role in computing, the mechanisms governing file allocation, directory traversal, and inode management are rarely made visible to learners in an interactive or accessible manner. Traditional OS curricula present these concepts through textbook diagrams and theoretical descriptions, leaving a significant experiential gap between understanding and application.

TriCoreFS addresses this gap through a dual-module architecture. The first module, the File Organiser, provides a practical demonstration of directory structures and file metadata by automatically categorizing uploaded folders into typed subdirectories. This module grounds abstract concepts—recursive traversal, file extension parsing, directory creation—in a tangible, immediately useful utility. Users interact with a disorganized folder and observe, in real time, how the system reads file metadata and constructs a logical hierarchical structure mirroring principles found in real operating systems. The File Organiser is additionally available as a globally installable NPM CLI tool (`npx smart-folder-organiser`), enabling the same organizational capabilities from the terminal.

The second module, the File System Simulator, shifts focus to the internal mechanics of disk storage. Users specify a file name and size, select from five allocation strategies (Contiguous, Linked, Indexed, UNIX Inode, and FAT-style), and receive a complete visualization of block allocation, inode structure, free space, and fragmentation. The simulator models UNIX-style inode design including direct, single-indirect, and double-indirect block pointers, providing exposure to the same structures found in production file systems such as ext4. Notably, Indexed allocation and UNIX Inode are treated as distinct strategies: the former uses a single-level index block, while the latter implements the full multi-level hybrid pointer model.

Together, the two modules embody three educational objectives:

1) Demonstrating the practical consequences of file system design on everyday file management.
2) Providing visual and interactive modeling of allocation strategies and their trade-offs.
3) Enabling direct comparison of fragmentation behavior, access speed, and memory overhead across strategies.

TriCoreFS is designed for students, educators, and OS enthusiasts seeking an experiential complement to theoretical study.

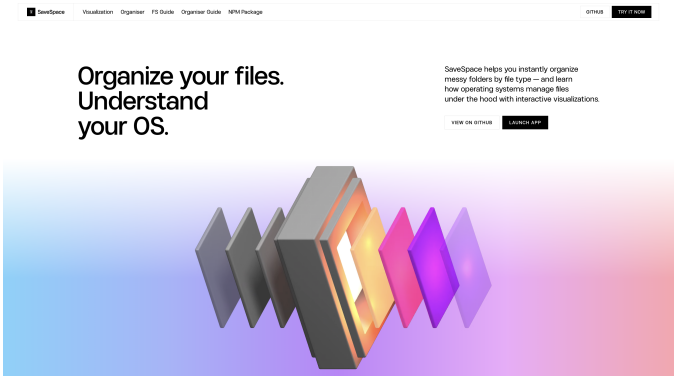

Fig. 1. Homepage

II. Problem Definition and Motivation

A. Core Problem

The study of operating system file systems presents a persistent pedagogical challenge: the concepts are inherently abstract and invisible during normal system use. Students learning about inode structures, block allocation tables, and fragmentation rarely encounter a direct, hands-on mechanism to observe these phenomena in action. This abstraction gap manifests in two distinct but related problems.

First, at the practical level, users routinely accumulate disorganized file collections across their systems. The absence of automated, metadata-aware organization tools forces manual sorting, consuming time and obscuring the relationship between file metadata and directory structure—concepts that are fundamental to OS design. Second, at the educational level, existing tools for teaching file system internals are either overly simplistic, lacking visual fidelity, or require complex environment setup that impedes accessibility.

B. Motivation

TriCoreFS is motivated by the recognition that two parallel needs—practical file management and OS education—can be addressed by a unified system built on the same underlying principles. The following specific gaps drive the system's design:

- **Lack of Interactive Simulators:** Most OS textbooks present allocation strategies through static diagrams. No widely accessible browser-based tool allows students to dynamically create files, choose strategies, and observe the resulting block allocation and fragmentation.
- **Invisible Inode Mechanics:** While inode structures underpin every UNIX-based file system, students rarely see a live inode populated with real values derived from their own input. TriCoreFS generates and displays a complete inode for every simulated file.
- **Fragmentation Invisibility:** External and internal fragmentation are described theoretically but seldom demonstrated comparatively. TriCoreFS quantifies and visualizes fragmentation metrics across all five strategies for any given file size.
- **Disconnected Theory and Practice:** The File Organiser module grounds OS directory concepts in a real-world utility, making the connection between theoretical file system design and practical system behavior explicit and observable.

III. Literature Review and Related Work

A. File System Allocation Strategies

The three classical file allocation strategies—contiguous, linked, and indexed—are well established in operating systems literature [1]. Contiguous allocation, analogous to contiguous memory allocation, offers fast sequential and random access but suffers from external fragmentation and requires advance knowledge of file size [2]. Linked allocation resolves external fragmentation by scattering blocks and chaining them via pointers, but introduces pointer overhead and eliminates efficient random access. Indexed allocation consolidates block pointers into a dedicated index block, balancing access efficiency and flexibility, and forms the basis of UNIX inode design [3].

B. FAT File Systems

The File Allocation Table (FAT) architecture, originally developed for MS-DOS, implements a centralized table maintaining next-block pointers for all files simultaneously [5]. FAT variants (FAT12, FAT16, FAT32) remain widely used in removable storage contexts due to their simplicity and cross-platform compatibility. TriCoreFS simulates FAT-style allocation by modeling the central table structure and chain traversal mechanism, enabling comparison with inode-based approaches.

C. Inode Architecture

The inode structure originating in UNIX System V and refined in BSD UNIX forms the metadata backbone of modern UNIX-like file systems including ext2, ext3, ext4, and XFS [3]. Each inode stores file metadata and block pointers organized into direct pointers (typically 12), a single indirect pointer, a double indirect pointer, and a triple indirect pointer, enabling support for files ranging from small to very large [4]. TriCoreFS models direct and indirect pointer tiers as a distinct UNIX Inode strategy, computing pointer requirements dynamically based on user-specified file size, and separately models single-level Indexed allocation as its own strategy.

D. Educational File System Tools

Prior work in OS education tooling includes NACHOS [6], an instructional OS framework, and OSTEP simulation tools [**?**], which provide command-line file system exercises. TriCoreFS differentiates itself by offering a fully browser-based, graphical, interactive experience requiring no local installation, combining practical file organization with theoretical simulation in a single unified interface, and extending accessibility through a published NPM CLI tool.

IV. System Design and Architecture

A. Architectural Overview

TriCoreFS implements a three-layer architecture: (1) Frontend Interactive Interface, (2) Processing and Simulation Engine, and (3) Browser-Native File System API Layer. The system is designed to operate entirely within the browser for both modules. The File Organiser performs all directory reading and classification client-side using HTML5 browser APIs, with no server-side upload or processing required. The File System Simulator likewise executes entirely in the browser as a stateful JavaScript module.

1) Frontend Layer: The client interface is implemented using React with TypeScript, providing two distinct views corresponding to each module:

- File Organiser interface with drag-and-drop folder selection, recursive scan results rendered as a virtual DOM tree, and categorized output preview organized under a master `Organised` folder
- File System Simulator interface with file creation form, strategy selector (five strategies), disk block visualizer with grid/list view toggle, inode panel, and fragmentation metrics dashboard

2) Simulation Engine: The core simulation logic executes in the browser as a JavaScript module implementing:

- Block calculation engine computing required blocks from file size and block size
- Five allocation strategy implementations with distinct block assignment algorithms (Contiguous, Linked, Indexed, UNIX Inode, and FAT)
- Inode constructor generating metadata and pointer structures per UNIX specification
- Fragmentation analyzer computing internal and external fragmentation metrics
- Comparison engine generating cross-strategy trade-off summaries
- Persistent state manager serializing and restoring disk and file system state via browser `localStorage`

3) Browser-Native File System API Layer: The File Organiser module operates entirely client-side using browser-native APIs:

- `webkitdirectory` attribute enabling recursive folder selection from the local file system
- `FileSystemDirectoryEntry` API for recursive directory traversal and path preservation
- File metadata extractor parsing MIME types and extensions in memory
- Virtual DOM tree constructor rendering the categorized output hierarchy without server interaction

B. Module 1: File Organiser — Design

The File Organiser implements a pipeline of four sequential operations corresponding to fundamental OS file system operations: scan, classify, construct, and relocate. All operations are performed locally in the browser without uploading files to any server.

Scan: The system performs a recursive depth-first traversal of the selected directory tree using the `FileSystemDirectoryEntry` API, enumerating all files and extracting metadata including file name, extension, size, and MIME type. This mirrors the recursive directory traversal operations performed by OS utilities such as `find` on UNIX systems.

Classify: Each file is mapped to a category based on its extension. The classification schema includes: `Images/` (jpg, png, gif, svg, webp), `PDFs/` (pdf), `Videos/` (mp4, mkv, avi, mov), `Documents/` (doc, docx, txt, ppt, xls), `Audio/` (mp3, wav, aac, flac, ogg), `Code/` (py, js, ts, c, cpp, java, html, css), `Archives/` (zip, tar, gz, rar, 7z), and `Others/` for unrecognized extensions.

Construct: Category directories are assembled under a single master parent folder named `Organised/`, demonstrating hierarchical directory creation analogous to nested `mkdir` system calls. This aggregation prevents clutter at the root level of the output and mirrors OS conventions for organizing categorized data under a unified namespace.

Relocate: Files are virtually moved to their respective category directories within the `Organised/` tree, demonstrating file relocation operations analogous to the `rename`/`mv` system calls at the OS level. The result is rendered as an interactive virtual directory tree in the browser.

C. Module 2: File System Simulator — Design

The simulator models a virtual disk partitioned into fixed-size blocks of configurable size (default 4 KB). Given a user-specified file size S and block size B, the number of required blocks N is computed as:

$$N = \left\lceil \frac{S}{B} \right\rceil \tag{1}$$

The internal fragmentation F_i of the last block is:

$$F_i = (N \times B) - S \tag{2}$$

Each of the five allocation strategies receives the value of N and implements a distinct block assignment algorithm as described in Section V. The virtual disk state—including block occupancy map, inode table, and strategy parameters—is serialized to browser `localStorage` after each operation, ensuring that the simulated file system persists across page reloads, mirroring the durability semantics of real-world persistent storage.

D. State Persistence Design

A key architectural feature of the File System Simulator is its use of browser `localStorage` for state persistence. After every allocation operation, the complete disk state (block map, inode entries, strategy metadata) is serialized to JSON and stored locally. On subsequent page loads, the state is deserialized and the visualization is restored. This design directly parallels the persistence model of physical file systems, where data survives power cycles through non-volatile storage.

V. Implementation Details

A. Contiguous Allocation

The contiguous allocator scans the simulated disk for a contiguous run of N free blocks beginning at the lowest available address. The algorithm proceeds as follows:

Scan free block list for contiguous run of length N
if run found at start block s **then**
  Allocate blocks s through $s + N - 1$
  Record start block and length in inode
  Mark blocks as occupied in disk map
else
  Report external fragmentation failure
end if

This strategy provides $O(1)$ random access using the formula block address $= s +$ offset, but external fragmentation accumulates as files are created and deleted over time. The simulator visualizes contiguous blocks as a highlighted sequential region on the disk map and reports cases where allocation fails due to fragmentation despite sufficient total free space.

B. Linked Allocation

The linked allocator assigns blocks from any available free location, embedding a pointer to the next block within each allocated block:

Initialize head pointer = NULL
for each of N required blocks **do**
  Select next available free block b
  Write pointer to b in previous block's pointer field
  Mark b as occupied
end for
Record head block in inode

The simulator displays each block with an embedded arrow indicating its successor, forming a visual chain. While external fragmentation is eliminated, the pointer field within each block constitutes pointer overhead, and sequential traversal is required to access any block at position k, yielding $O(k)$ access time.

C. Indexed Allocation

The indexed allocator assigns one dedicated index block containing pointers to all N data blocks. This is modeled as a flat, single-level structure in which all data block addresses are stored in a single index block allocated alongside the file data. This approach provides $O(1)$ random access (given the index block address) while eliminating external fragmentation. The index block itself constitutes a fixed overhead regardless of file size, making this strategy most efficient for medium-sized files.

D. UNIX Inode Allocation

The UNIX Inode strategy implements the full multi-level pointer hierarchy as found in production UNIX-like file systems. For files exceeding the direct pointer capacity of the inode, the system extends to single-indirect and double-indirect pointer blocks:

- **Direct pointers:** 12 pointers stored directly in the inode, supporting files up to $12B$ bytes
- **Single indirect:** One pointer to a block containing $B/4$ pointers (assuming 4-byte pointers), supporting files up to $(12 + B/4)B$ bytes
- **Double indirect:** One pointer to a block of pointers to pointer blocks, extending capacity significantly

The inode panel in the simulator displays all populated fields including file name, size, block count, allocation strategy, creation timestamp, and the full pointer structure with computed values. This strategy is distinct from single-level Indexed allocation in that it dynamically selects the appropriate pointer tier based on file size, matching the behavior of real ext4 inodes.

E. FAT-Style Allocation

The FAT simulator maintains a central table array indexed by block number. Each entry contains either the index of the next block in the chain, a terminal marker (`EOF`), or a free marker (`FREE`):

Initialize FAT table with all entries = FREE
Allocate N blocks from free list
for $i = 0$ to $N - 2$ **do**
  FAT[block[i]] = block[$i + 1$]
end for
FAT[block[$N - 1$]] = EOF
Record head block in directory entry

The simulator renders the FAT table alongside the disk visualization, highlighting active chain entries and showing the traversal path from head to EOF.

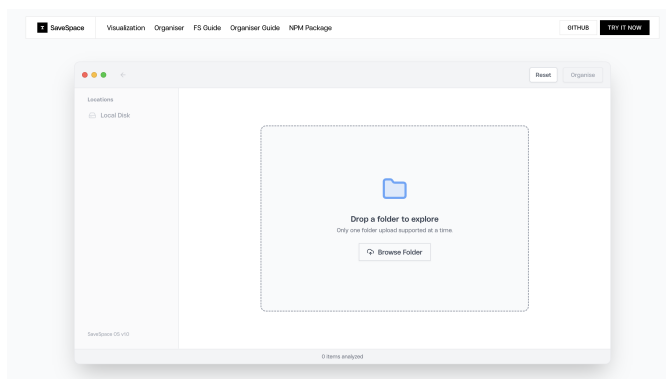

Fig. 2. Organiser

F. Inode Constructor

For every simulated file regardless of allocation strategy, the system constructs and displays a complete inode record:

- **fileId:** Simulated inode number
- **fileName:** User-specified name
- **fileSize:** Size in bytes
- **blockCount:** Computed value N

- **allocationStrategy:** Selected strategy
- **directPointers:** Up to 12 block addresses
- **singleIndirect:** Pointer block address (if required)
- **doubleIndirect:** Double pointer block address (if required)
- **createdAt:** Timestamp
- **permissions:** Simulated UNIX permission bits

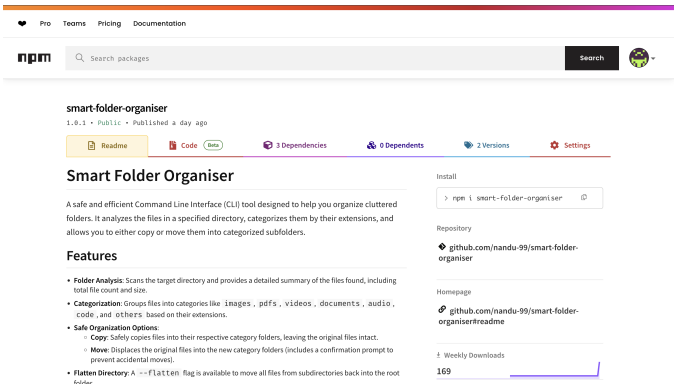

Fig. 3. NPM Package

G. NPM CLI Tool

The File Organiser functionality is additionally packaged as a globally executable NPM CLI tool, invocable via `npx smart-folder-organiser`. The CLI tool extends the web module's capabilities with features tailored to terminal-based OS workflows:

- **Folder Size Analysis:** Before organizing, the tool analyzes and reports the size distribution of files within the target directory, aiding users in understanding storage consumption—a concept tied directly to OS disk management.
- **Copy and Move Modes:** Users may choose between `Copy` (non-destructive) and `Move` operations, preventing accidental data loss while demonstrating the OS-level distinction between `cp` and `mv` system calls.
- **Flatten Mode:** A `--flatten` flag implements an inverse operation that extracts files from nested subdirectories back into the root of the target folder, mirroring the behavior of recursive directory collapsing in OS file management.
- **Interactive Terminal UI:** The tool uses `inquirer` for interactive prompts, `picocolors` for terminal output formatting, and `cli-progress` for real-time visual progress bars, together delivering a polished terminal OS-style interface.

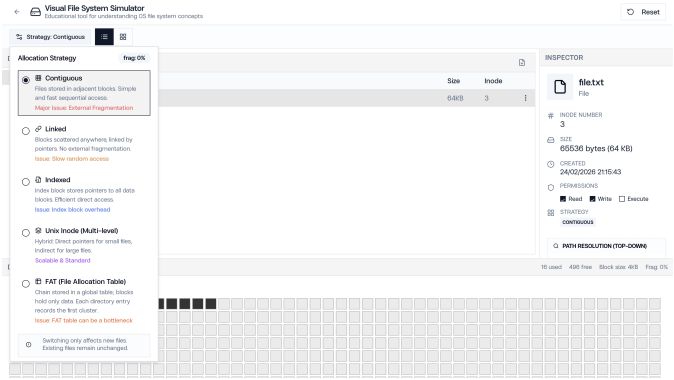

Fig. 4. Visualisation

VI. UI/UX and Educational Enhancements

A. Interactive Guide Routes

TriCoreFS includes dedicated in-application guide routes to scaffold the user's theoretical understanding alongside the interactive tools:

- **/guide:** A general simulator guide walking users through the five allocation strategies, explaining their trade-offs and directing users toward relevant simulation scenarios.
- **/organiser-guide:** A dedicated guide that breaks down core OS concepts—including Virtualization and Recursive Directory Traversal—within the context of the File Organiser module. This route grounds theoretical principles in the practical behavior users observe in the organiser, creating a direct feedback loop between concept and application.

B. Modern Simulation UI

1) Grid and List View Toggle: The disk block visualizer supports a toggle between Grid View and List View display modes, simulating the view switching behavior common in desktop OS file explorers such as macOS Finder and Windows Explorer. This feature reinforces the connection between the simulator and familiar OS UI paradigms while improving usability for different screen sizes and use cases.

2) Immersive 3D Landing Components: Responsive `Spline` 3D components are integrated on the application's landing pages, providing visually engaging representations of file system structures. These components serve an educational priming function, offering users an intuitive spatial metaphor for abstract concepts such as block grids and directory trees before they engage with the interactive simulators.

VII. Performance Analysis

A. Allocation Strategy Comparison

Table I presents a comprehensive comparison of the five allocation strategies across key performance and overhead dimensions as modeled in TriCoreFS.

TABLE I
ALLOCATION STRATEGY COMPARISON

Strategy	Ext. Frag	Int. Frag	Access Speed	Memory Overhead
Contiguous	Yes	Yes	Fast $O(1)$	Low
Linked	No	Yes	Slow $O(n)$	Pointer per block
Indexed	No	Yes	Fast $O(1)$	Index block
UNIX Inode	No	Yes	Fast $O(1)$	Tiered pointer blocks
FAT	No	Yes	Medium $O(n)$	Full table

B. Internal Fragmentation Analysis

All strategies exhibit internal fragmentation in the final allocated block. For a file of size S and block size B, the wasted space in the last block is $(N \times B) - S$ where $N = \lceil S/B \rceil$. The simulator computes and displays this value explicitly, allowing users to observe how block size selection affects storage efficiency.

C. External Fragmentation Simulation

The contiguous allocator demonstrates external fragmentation by simulating a disk state with sufficient total free space but no contiguous run long enough to satisfy the allocation request. The simulator visualizes the fragmented free regions and reports the allocation failure, concretizing a concept that is otherwise purely theoretical.

D. File Organiser Performance

The File Organiser processes selected directories through a single-pass recursive scan using the browser-native `FileSystemDirectoryEntry` API, achieving $O(n)$ time complexity where n is the total number of files. Classification via extension lookup operates in $O(1)$ per file using a hash map. The virtual DOM tree is constructed in memory without any filesystem I/O overhead on the server side, as all operations are local to the browser.

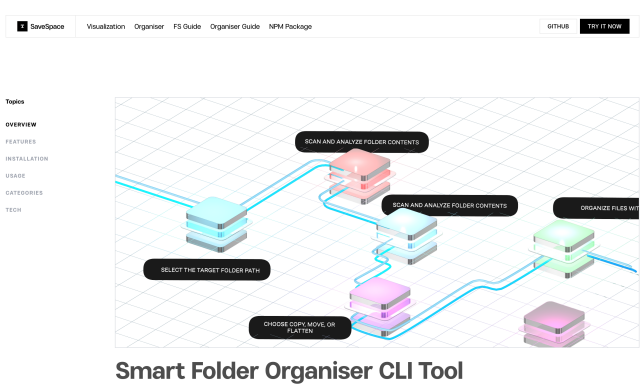

Fig. 5. Guide Pages

VIII. KEY CHALLENGES AND SOLUTIONS

A. Browser-Based Folder Access

Challenge: Standard HTML file inputs do not expose directory structure or recursive file trees, complicating folder traversal in a web application.

Solution: The File Organiser leverages the `webkitdirectory` attribute on the file input element combined with the `FileSystemDirectoryEntry` API, enabling recursive directory reading with full path preservation entirely client-side. Relative paths are parsed to reconstruct the original directory hierarchy and render it as a virtual tree in the browser, requiring no server upload.

B. Dynamic Inode Generation

Challenge: Generating a meaningful, correctly structured inode with accurate direct and indirect pointer counts requires dynamic computation based on file size and block size inputs that vary per user interaction.

Solution: The inode constructor implements a tiered pointer assignment algorithm that first fills direct pointer slots, then overflows to single-indirect, and subsequently to double-indirect pointer blocks, computing all values analytically from the input parameters without requiring an actual disk.

C. Fragmentation Visualization

Challenge: Rendering a disk block map that clearly distinguishes allocated blocks, free blocks, index blocks, pointer blocks, and fragmented regions in a compact, readable interface.

Solution: The disk visualizer assigns distinct color codes to each block state and renders the disk as a responsive grid, with a toggle to switch to list view for detailed inspection. Strategy-specific block types (index blocks for indexed allocation, FAT entries for FAT simulation, pointer tier blocks for UNIX Inode) are rendered with distinct visual treatments and tooltips displaying block metadata.

D. Cross-Strategy State Management

Challenge: Allowing users to switch between allocation strategies for the same file while maintaining consistent disk state and preventing stale visualization artifacts.

Solution: Each strategy change triggers a complete disk state reset and reallocation pass. React state management ensures the visualization, inode panel, and metrics dashboard re-render atomically upon strategy selection. The resulting state is immediately persisted to `localStorage` to maintain consistency across sessions.

E. CLI Tool Data Safety

Challenge: Providing a file organization CLI tool that operates on live user directories without risking accidental data loss, a critical concern when implementing OS-level `mv` semantics.

Solution: The CLI tool exposes an explicit choice between `Copy` and `Move` modes at runtime via `inquirer` interactive prompts. Move operations are performed only upon explicit

user confirmation, and the `--flatten` flag is similarly gated behind a confirmation step. Real-time progress is reported via `cli-progress` bars, giving users visibility into the operation before it completes.

IX. Security Considerations

A. Client-Side File Access Scope

The File Organiser accesses files exclusively through the browser's sandboxed File System API. No file content is transmitted to any server; all processing occurs locally within the browser's security sandbox. The `webkitdirectory` API grants read-only access scoped to the user-selected folder, preventing access to any files outside that scope.

B. Input Validation

User inputs to the File System Simulator (file size, block size, file name) are validated on the client side to prevent injection attacks and ensure simulation parameters remain within defined ranges. File size inputs are capped at a configurable maximum to prevent denial-of-service through computationally expensive simulation requests.

C. Sandboxed Simulation

The File System Simulator operates entirely within JavaScript's browser sandbox, performing no actual disk operations. All simulated disk state is maintained in memory and in browser `localStorage`, with no server-side persistence surface. The `localStorage` store is scoped to the application origin, preventing cross-site data access.

X. Future Enhancements

A. Triple Indirect Pointer Support

Extend the UNIX Inode model to include triple indirect pointers, enabling simulation of very large files and providing complete coverage of the UNIX inode specification.

B. Disk Scheduling Simulation

Incorporate disk scheduling algorithms (FCFS, SSTF, SCAN, C-SCAN) to complement the allocation simulator, allowing users to observe how block placement interacts with seek time optimization.

C. Multi-File Simulation

Extend the simulator to support multiple concurrent files on the same virtual disk, demonstrating inter-file fragmentation dynamics, inode table management, and directory entry structures.

D. File Deletion and Defragmentation

Implement file deletion operations and a defragmentation tool, allowing users to observe how free space is reclaimed and how contiguous allocation performance degrades and recovers over a sequence of create and delete operations.

E. Extended File Type Support

Expand the File Organiser's classification schema to support user-defined category rules, enabling custom organization workflows beyond the default extension-based taxonomy.

F. Export and Reporting

Generate downloadable simulation reports summarizing inode structure, block allocation map, and fragmentation metrics for use in academic assignments and documentation.

XI. Conclusion

TriCoreFS successfully demonstrates that the core concepts of operating system file systems—block allocation, inode structure, fragmentation, and directory management—can be made accessible and interactive through a well-designed dual-module educational platform. The File Organiser module provides a practical, real-world application of directory traversal and file metadata concepts, operating entirely client-side using browser-native APIs and extended by a published NPM CLI tool (`npx smart-folder-organiser`) for terminal workflows. The File System Simulator delivers visual, hands-on exploration of five allocation strategies—Contiguous, Linked, Indexed, UNIX Inode, and FAT—with persistent state across sessions via `localStorage`, immersive 3D landing visuals, and in-application guide routes that bridge theory and practice.

Key contributions of this work include: (1) a unified platform connecting practical file management with theoretical OS concepts; (2) a dynamic UNIX-style inode generator producing accurate tiered pointer structures from user input, distinct from the flat Indexed strategy; (3) a comparative fragmentation analysis framework enabling direct observation of trade-offs across five strategies; (4) a browser-native, server-free architecture requiring zero installation overhead for the web modules; and (5) a CLI tool extending organizational capabilities to terminal environments with safe Copy/Move modes and directory flattening.

The system is architecturally extensible, with clear pathways to incorporate disk scheduling simulation, multi-file interaction, and full triple-indirect pointer modeling. As operating system education increasingly demands experiential learning tools, TriCoreFS demonstrates that interactive browser-based simulators can effectively bridge the gap between abstract specification and concrete understanding.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.

[2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Upper Saddle River, NJ: Pearson, 2014.

[3] M. J. Bach, *The Design of the UNIX Operating System*. Englewood Cliffs, NJ: Prentice Hall, 1986.

[4] R. Love, *Linux Kernel Development*, 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2010.

[5] Microsoft Corporation, "FAT32 File System Specification," Microsoft Extensible Firmware Initiative FAT32 File System Specification, Aug. 2005.

[6] T. E. Anderson, "The Case for Application-Specific Hardware," in *Proc. 2nd Symp. on Operating Systems Design and Implementation (OSDI)*, 1996.